

Práctica 5 - Ejercicios 1.5

Verificación de los mecanismos de integración de los lenguajes de marcas con los lenguajes de programación de clientes Web.

Actividad 1 - Código en ficheros externos separados

Se crean `index.html` y `script.js` en la misma carpeta con el código de ficheros separados del apartado B.

El archivo `index.html` solo se ocupa de la estructura y enlaza el fichero externo:

```
index.html

<!DOCTYPE html>
<html>
<head>
  <title>Myfpschool</title>
  <!-- Enlace al fichero script.js ubicado en la misma carpeta -->
  <script src="script.js"></script>
</head>
<body>
</body>
</html>
```

El archivo `script.js` contiene toda la lógica y la ejecuta al cargarse:

```
script.js

// Definición de una función básica de saludo
function diAlgo() {
  alert("hola"); // Muestra un cuadro modal emergente con el texto "hola"
}

// Invocación directa de la función al cargarse el fichero
diAlgo();
```

Al abrir la página en el navegador aparece el cuadro de alerta con el texto "hola":

```
index.html

hola

Aceptar
```

El fichero `script.js` se descarga y se ejecuta al cargar la página. Para verlo se sustituyó el `alert` nativo por una función que guarda el texto que recibe, sin modificar `script.js`:

Comprobación de la actividad 1

El fichero `script.js` se enlaza desde `index.html` y se ejecuta al cargar la página:

Se ha ejecutado y ha llamado a `alert()` con el texto: "hola"

Como se ve, `script.js` se ha ejecutado y ha llamado a `alert()` con el texto "hola".

Actividad 2 - Código JavaScript embebido dentro del HTML

El mismo aviso, pero con el código incrustado en el propio documento. La función se declara en el `head` y se invoca desde un segundo bloque `script` en el `body`:

```
index.html

<!DOCTYPE html>
<html>
<head>
  <title>Myfpschool</title>
  <script>
    // Se declara la función dentro de la cabecera head
    function diAlgo() {
      alert("Hola");
    }
  </script>
</head>
<body>
  <!-- Se ejecuta la función en el cuerpo del documento body -->
  <script>
    diAlgo();
  </script>
</body>
</html>
```

El efecto ante el usuario es el mismo, con el cuadro de alerta:

```
index.html

Hola

Aceptar
```

El resultado es idéntico al de la actividad 1. La diferencia está en el mantenimiento: aquí el código queda disperso entre el `head` y el `body`, mientras que en la actividad 1 toda la lógica vive en un único archivo.

Práctica de laboratorio guiada - Reorganización del proyecto

1. Estructura de carpetas

Se organiza el proyecto separando cada recurso en su propio directorio:

Estructura de carpetas

```
mi_proyecto/  
├── css/  
│   └── estilos.css  
├── js/  
│   └── logica.js  
└── index.html
```

2. Traslado del script embebido a `js/logica.js`

El bloque de script que cambiaba el color del botón se traslada a `js/logica.js`:

mi_proyecto/js/logica.js

```
// Antes este bloque estaba embebido dentro de index.html.  
// Ahora vive en la carpeta js/ y se enlaza desde el head con la ruta relativa.  
function cambiarColor() {  
  const boton = document.getElementById('miBoton');  
  boton.style.backgroundColor = 'green';  
  document.getElementById('estado').innerHTML = 'El color del botón ha cambiado a verde';  
}  
  
// El script se ejecuta al cargar la página, cuando el DOM ya está construido.  
document.getElementById('miBoton').addEventListener('click', cambiarColor);
```

Los estilos se separan también en `css/estilos.css`:

mi_proyecto/css/estilos.css

```
/* Hoja de estilos del proyecto: separamos el CSS del HTML y del JS */  
body {  
  font-family: "DejaVu Sans", sans-serif;  
  margin: 30px;  
}  
  
h1 {  
  color: #12395c;  
}  
  
#miBoton {  
  background-color: #12395c;  
  color: #ffffff;  
  border: none;  
  padding: 10px 18px;  
  font-size: 15px;  
  cursor: pointer;  
}
```

Y `index.html` queda limpio, enlazando cada recurso con su ruta relativa:

mi_proyecto/index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Proyecto reorganizado</title>
  <!-- La hoja de estilos va en su propia carpeta -->
  <link rel="stylesheet" href="./css/estilos.css">
  <!-- El script se enlaza desde el head con la ruta relativa correcta.
       Con "defer" se descarga en segundo plano y espera a que el HTML
       esté parseado, así el script ya encuentra los nodos del body. -->
  <script defer src="./js/logica.js"></script>
</head>
<body>
  <h1>Proyecto reorganizado</h1>
  <p id="estado">El botón sigue con su color original</p>
  <button type="button" id="miBoton">Cambiar color</button>
</body>
</html>
```

El script se enlaza con el atributo `defer`. Sin él, el navegador ejecuta el script del `head` antes de haber construido el `body`, así que `document.getElementById('miBoton')` devolvería `null` y el código fallaría. Con `defer` la ejecución se retrasa hasta que el documento HTML se ha parseado por completo.

Al pulsar el botón, su color cambia a verde:

Proyecto reorganizado

El color del botón ha cambiado a verde

Cambiar color

3. Comprobación en la pestaña Network

La pestaña Network de las herramientas del desarrollador (F12) muestra que los dos ficheros se sirven correctamente:

Consola del navegador

Filtrar: JS/CSS			
Network	Estado	Tipo	Iniciador
logica.js	200 OK	script	mi_proyecto/index.html
estilos.css	200 OK	stylesheet	mi_proyecto/index.html

Ejercicios del 1.2 y 1.4 con la estructura anterior

Los siguientes ejercicios repiten los de los apartados 1.2 y 1.4 con esta misma organización de carpetas.

Del apartado 1.2

Con el contenido, el encabezado y los saludos en distintos idiomas, resueltos desde `js/logica.js`:

mi_proyecto/index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejercicios del 1.2 con estructura de carpetas</title>
  <link rel="stylesheet" href="./css/estilos.css">
  <script defer src="./js/final-12-logica.js"></script>
</head>
<body>
  <h1 id="titulo">Modificando el código HTML</h1>
  <p id="prueba">Modificando el contenido.</p>

  <button type="button" id="btnCambiar">¡Dale!</button>

  <p id="saludo"></p>
  <select id="idioma">
    <option value="es">Español</option>
    <option value="en">Inglés</option>
    <option value="fr">Francés</option>
  </select>
  <button type="button" id="btnSaludar">Saludar</button>
</body>
</html>
```

mi_proyecto/js/logica.js

```
// Ejercicio del 1.2 resuelto con la estructura de carpetas del 1.5.
// La lógica ya no vive dentro de index.html, sino en js/logica.js.
function cambiarContenido() {
  document.getElementById('titulo').innerHTML = 'CAMBIANDO EL TITULO';
  document.getElementById('prueba').innerHTML = 'CAMBIANDO el contenido!';
}

function saludar(idioma) {
  if (idioma === 'es') return 'Hola';
  if (idioma === 'en') return 'Hello';
  return 'Bonjour';
}

function mostrarSaludo() {
  const elegido = document.getElementById('idioma').value;
  document.getElementById('saludo').innerHTML = saludar(elegido);
}

document.getElementById('btnCambiar').addEventListener('click', cambiarContenido);
document.getElementById('btnSaludar').addEventListener('click', mostrarSaludo);
```

Al pulsar los dos botones cambian el título y el párrafo, y se muestra el saludo del idioma elegido:

CAMBIANDO EL TITULO

CAMBIANDO el contenido!

¡Dale!

Hola

Español ▾

Saludar

Del apartado 1.4

Con las funciones de formatear fecha en español y obtener el último día del mes, también en `js/logica.js`:

mi_proyecto/index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejercicios del 1.4 con estructura de carpetas</title>
  <link rel="stylesheet" href="./css/estilos.css">
  <script defer src="./js/final-14-logica.js"></script>
</head>
<body>
  <h1>Ejercicios de fechas</h1>
  <button type="button" id="btnFechas">Mostrar fechas</button>
  <p id="salida"></p>
</body>
</html>
```

mi_proyecto/js/logica.js

```
// Ejercicios del 1.4 resueltos con la estructura de carpetas del 1.5.
function formatearFechaEspanola(fecha) {
  const dia = String(fecha.getDate()).padStart(2, '0');
  const mes = String(fecha.getMonth() + 1).padStart(2, '0');
  const anio = fecha.getFullYear();
  const hora = String(fecha.getHours()).padStart(2, '0');
  const minuto = String(fecha.getMinutes()).padStart(2, '0');
  return dia + '/' + mes + '/' + anio + ' ' + hora + ':' + minuto;
}

function obtenerUltimoDiaMes(anio, mes) {
  // El día 0 del mes siguiente es el último día del mes indicado
  return new Date(anio, mes, 0).getDate();
}

function mostrarFechas() {
  const ahora = new Date();
  document.getElementById('salida').innerHTML =
    'Fecha formateada: ' + formatearFechaEspanola(ahora) +
    '<br>Días de febrero de 2024: ' + obtenerUltimoDiaMes(2024, 2) +
    '<br>Días de febrero de 2023: ' + obtenerUltimoDiaMes(2023, 2);
}

document.getElementById('btnFechas').addEventListener('click', mostrarFechas);
```

Al pulsar el botón se muestran la fecha formateada y los días de febrero de dos años distintos:

Ejercicios de fechas

Mostrar fechas

Fecha formateada: 01/10/2026 20:54
Días de febrero de 2024: 29
Días de febrero de 2023: 28